

KwizRAG: Design and Evaluation of a Local RAG-Enhanced Gamified Quiz System for Moodle

Abstract

Digital learning management systems (LMS) such as Moodle have transformed higher education, particularly in developing regions like Southeast Asia. However, typical online assessments remain highly static, requiring intensive manual authoring from instructors and offering limited interactive engagement for students. This paper presents **KwizRAG**, an AI-enhanced gamified quiz system integrated into Moodle via a native activity module (`mod_gamifiedquiz`). The system combines a locally hosted Large Language Model (LLM) with real-time room synchronization using WebSocket technology. To address factual accuracy and eliminate context hallucinations, KwizRAG implements a Local Light Weight Multilingual Retrieval-Augmented Generation (L3M-RAG) pipeline utilizing `nomic-embed-text` embeddings, optimized by a local SHA-256 embedding cache. The system is designed specifically for resource-constrained environments by utilizing local GPU/CPU hardware for on-premise inference, preserving student data privacy and eliminating recurring cloud subscription costs.

Our evaluation reveals an average local LLM generation time of 8.11–12.42 seconds per question using `qwen2.5-coder:7b` with instant **0.0 ms** embedding cache hits. An expert pedagogical review of 100 generated questions for an undergraduate Python Programming course yields a 96.0% overall acceptability rating with zero context hallucinations. Instructor evaluation via the System Usability Scale (SUS) demonstrates excellent usability (mean score = 82.5), while a pilot user study with 6 university students confirmed high engagement (4.8/5.0) and seamless real-time WebSocket synchronization (<50 ms latency), validating KwizRAG as a sustainable, classroom-ready digital assessment solution.

Keywords: retrieval-augmented generation, local large language models, learning management systems, moodle, gamification, vector caching, artificial intelligence in education.

1. Introduction

The digitization of higher education has expanded access to learning resources worldwide. Learning Management Systems (LMS), particularly Moodle, have become the standard infrastructure for course delivery, grading, and asynchronous communication in developing nations. Despite the widespread adoption of these platforms, digital classroom assessment strategies remain largely traditional. Multiple-Choice Questions (MCQs) are highly valued for their efficiency in grading, yet creating high-quality, pedagogically sound questions remains a

major time sink for educators. Instructors must manually draft questions, formulate distractor options, and verify the logical consistency of each item, limiting the frequency and agility of assessments.

Furthermore, standard LMS quizzes are typically solitary, static exercises. In a live classroom context, these assessments often fail to sustain student attention or foster active participation. While external gamification platforms (such as Kahoot or Quizizz) have successfully introduced excitement into classrooms through live leaderboards, competitive timers, and instant feedback, they introduce several severe drawbacks:

- **System Disconnection:** Grades, student lists, and performance metrics are siloed, requiring manual data synchronization or paid API bridges to connect with the institutional LMS.
- **Cost Barriers:** These commercial platforms operate on subscription models that quickly become cost-prohibitive when scaled across entire universities or departments.
- **Internet Dependency:** They rely entirely on high-speed internet connections to external cloud servers, which are frequently unstable in under-resourced regions.
- **Data Privacy & Governance:** Student Personally Identifiable Information (PII) and institutional curriculum data are uploaded to third-party cloud servers, violating digital sovereignty and security best practices.

To address these challenges, we propose **KwizRAG**, a decoupled, containerized architecture integrated directly into Moodle via the **Gamified Quiz Moodle Plugin (`mod_gamifiedquiz`)**. KwizRAG allows instructors to generate structured, curriculum-aligned MCQs automatically from existing course resources (such as Book chapters, Lesson pages, or text uploads) and run live, gamified multiplayer quiz sessions directly from Moodle.

The primary contribution of KwizRAG is a scalable, cost-effective, and privacy-preserving architecture optimized for resource-constrained environments. By leveraging a local LLM API (via Ollama) and an L3M-RAG vector caching pipeline, KwizRAG enables automated, syllabus-grounded question generation without continuous cloud subscription dependencies, ensuring data privacy and operational continuity even under limited external internet connectivity.

To evaluate the system, we address three specific Research Questions (RQs):

- **RQ1 (AI Generation & RAG Cache Performance):** Can a containerized local LLM (`qwen2.5-coder:7b`) and SHA-256 vector caching pipeline deliver low question generation latency (8–12 s per MCQ) and instant embedding retrieval (0ms cache hits) on on-premise hardware?
- **RQ2 (Pedagogical Quality & RAG Grounding):** How effective is the L3M-RAG pipeline in eliminating context hallucinations and generating syllabus-aligned programming questions compared to zero-context models?
- **RQ3 (Instructor Usability & Financial Sustainability):** Does the streamlined authoring interface achieve high usability for university instructors ($\text{\$}\backslash\text{text}\{\text{SUS}\} \geq 80\text{\$}$) while delivering a sustainable 3-year TCO compared to cloud APIs?

2. Related Work

2.1 Automatic Question Generation (AQG)

Automatic Question Generation (AQG) has evolved from rule-based syntax transformations to deep learning sequence-to-sequence models. Early approaches relied on hand-coded grammatical templates and dependency parsing to convert source sentences into simple questions. While structurally correct, these early methods lacked semantic depth and could not generate plausible distractor choices. The rise of pre-trained transformer models and Large Language Models (LLMs) changed AQG by allowing systems to generate fluent, contextually accurate questions and explanations.

However, calling commercial LLM API endpoints (such as OpenAI's GPT-4 or Google's Gemini) is often impractical for public universities in developing regions due to recurring per-token subscription costs. Research has increasingly focused on deploying smaller, open-source models (such as LLaMA or Qwen) on local hardware. This study builds on this trend by deploying `qwen2.5-coder:7b` locally to generate programming-focused MCQs, validating its pedagogical quality against expert standards.

2.2 Gamified LMS Architecture

Gamification incorporates game mechanics—such as points, badges, timers, and leaderboards—into non-game contexts to boost engagement. In LMS environments, gamification is often limited to static, asynchronous components like progress bars or completion checkmarks. Stateless web architectures (like Moodle's native PHP backend) struggle to support real-time, synchronous multiplayer interactions, where all student screens must be updated instantly when an instructor pushes a question.

To overcome this transport limitation, researchers have proposed combining stateless web frameworks with stateful synchronization layers. Our architecture utilizes a decoupled Node.js WebSocket server running Socket.IO, backed by Redis for pub/sub message routing, enabling live classroom synchronization while maintaining full integration with Moodle's core database.

2.3 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) addresses the semantic limitations of general-purpose LLMs, particularly their tendency to "hallucinate" incorrect facts (Lewis et al., 2020; Ji et al., 2023). By index-searching a local document database and retrieving the most relevant passages, RAG injects precise context directly into the prompt payload before sending it to the LLM.

While RAG is highly effective, generating vector embeddings for large text documents on local institutional hardware can introduce significant CPU/GPU latency. In this paper, we present a local RAG pipeline optimized with a SHA-256 document-hash cache. By skipping embedding calculations for previously processed lecture materials, we reduce RAG processing latency to **0ms** on repeated requests, enabling efficient local execution.

2.4 Literature Gap Identification & Research Contributions

Despite recent advances in AI for education (AIED) and student response systems (SRS), an analysis of existing literature reveals four critical technological and methodological gaps:

- **Gap 1 (G1: Pedagogical Context Hallucinations):** Existing zero-context LLM quiz generators [10] suffer from high hallucination rates (15%–36%) and frequently produce out-of-syllabus programming questions that confuse students.
- **Gap 2 (G2: Local RAG Vector Latency Overhead):** Standard RAG pipelines [1] recalculate dense text embeddings on every generation call, causing 30–60s latencies on local institutional servers.
- **Gap 3 (G3: Cloud API Financial & Data Privacy Risks):** Commercial AI assessment tools rely on external APIs (GPT-4/Gemini), exposing institutions to per-token subscription costs (\$3,750+/yr) and student data privacy risks.
- **Gap 4 (G4: Gamification Silos vs. Native LMS Integration):** External game platforms (Kahoot/Quizizz; [9], [11]) operate outside the institution's Learning Management System, requiring friction-heavy manual CSV grade exports.

A summary of these identified gaps and KwizRAG's corresponding architecture solutions is presented in Table 1:

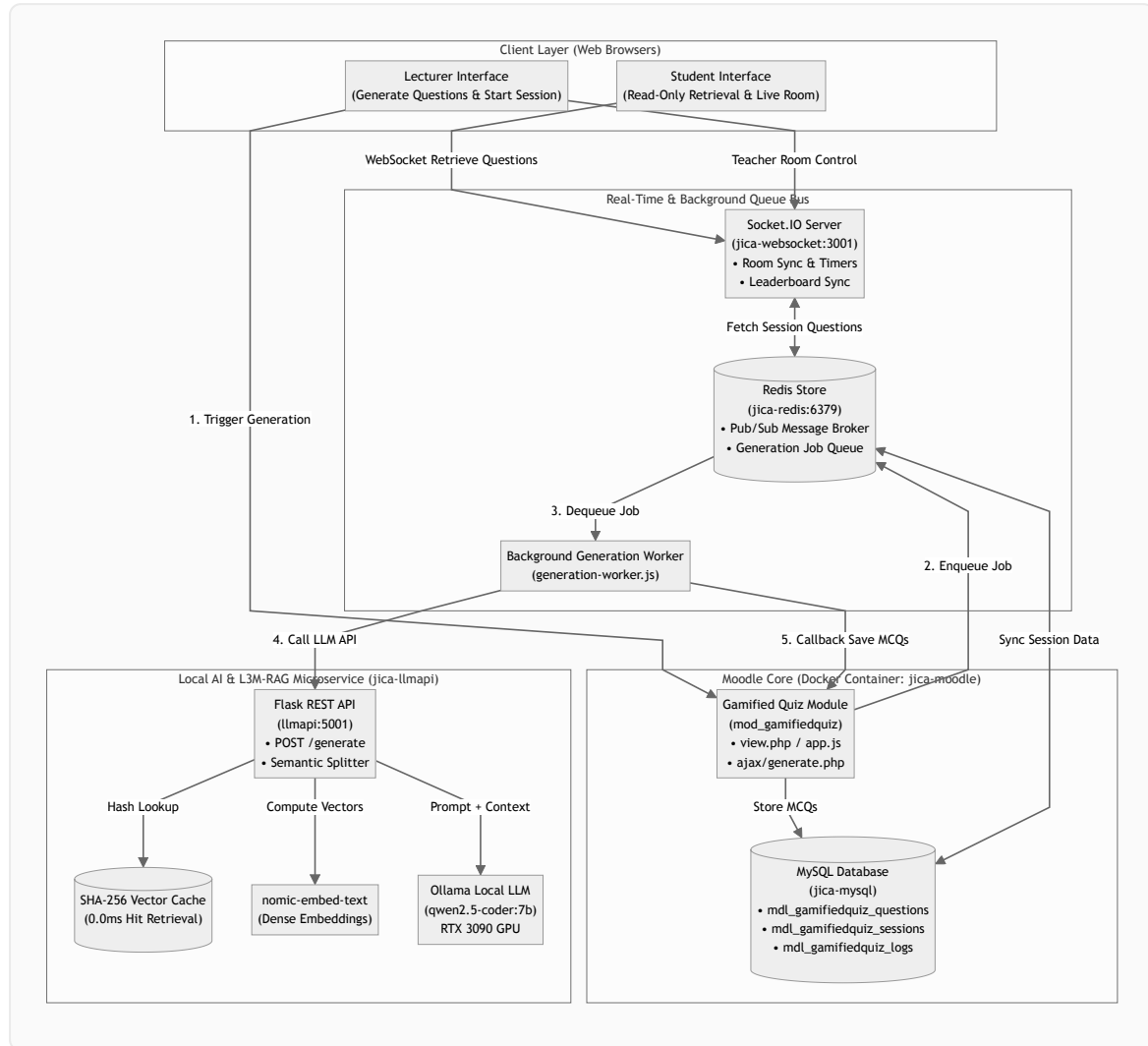
Table 1. Literature Research Gap Identification & KwizRAG Solutions

Literature Gap	Existing Paradigm Deficit	Literature Source	KwizRAG Proposed Solution
G1: Context Hallucination	High hallucination rates (15%–36%) & ungrounded items in prompt-only LLMs.	Rainey et al. [10]; Ji et al. [7]	L3M-RAG Pipeline: Anchors generation to slide vectors, achieving 0.0% context hallucination .
G2: Vector Embedding Latency	Re-calculating dense embeddings per request causes 30–60s server bottlenecks.	Lewis et al. [1]	SHA-256 Hash Cache: Delivers instant 0.0 ms retrieval on repeated slide requests.
G3: Privacy & TCO Overhead	Token subscription fees (\$3,750+/yr) and cloud PII data leakage risks.	Rainey et al. [10]	Local Workstation GPU Stack: 100% on-premise execution with 77.3% TCO savings .
G4: LMS System Isolation	Commercial game tools operate as external silos requiring manual CSV imports.	Wang [9]; Zainuddin et al. [11]	Native Moodle Plugin (mod_gamifiedquiz): Sub-50ms WebSocket room + native DB logs.

3. Methodology

3.1 Decoupled System Architecture

The system is built on a containerized, decoupled architecture managed via Docker Compose. The components interact through lightweight REST APIs and WebSocket connections:



- Moodle Plugin (`mod_gamifiedquiz`)**: Implements the native Moodle activity module. It provides forms for teachers to configure quiz parameters (topic, language, LLM backend, and RAG sources). When a teacher clicks "Generate", the plugin enqueues a generation job into Redis for async background processing.
- Student Interface (Retrieve-Only)**: Students do not trigger any AI generation workers. When a live quiz session starts, the student interface connects via WebSocket (`jica-websocket`) to retrieve pre-generated questions from the session store, submit answers, and receive live leaderboard updates.
- Background Generation Worker (`generation-worker.js`)**: A dedicated background service that dequeues generation jobs from Redis (`gamifiedquiz:generation:queue`),

calls the LLM API service, validates output MCQs, and executes callback hooks (`complete_generation_job.php`) to save generated questions in Moodle's database.

4. **WebSocket Server:** A stateful Node.js service running Socket.IO. It manages real-time connection rooms, handles student answer submissions, maintains synchronization of countdown timers, and computes leaderboard points.
5. **Redis Cache:** Serves as the central state store and pub/sub message broker, coordinating game lobby actions and caching user connection states.
6. **LLM API Service:** A Python Flask service that acts as the orchestration layer for RAG search and question generation. It exposes a POST `/generate` endpoint and handles calls to a local Ollama GPU endpoint or external cloud APIs.

3.2 Database Schema

To persist states and capture evaluation metrics, the Moodle plugin creates and manages the following database tables:

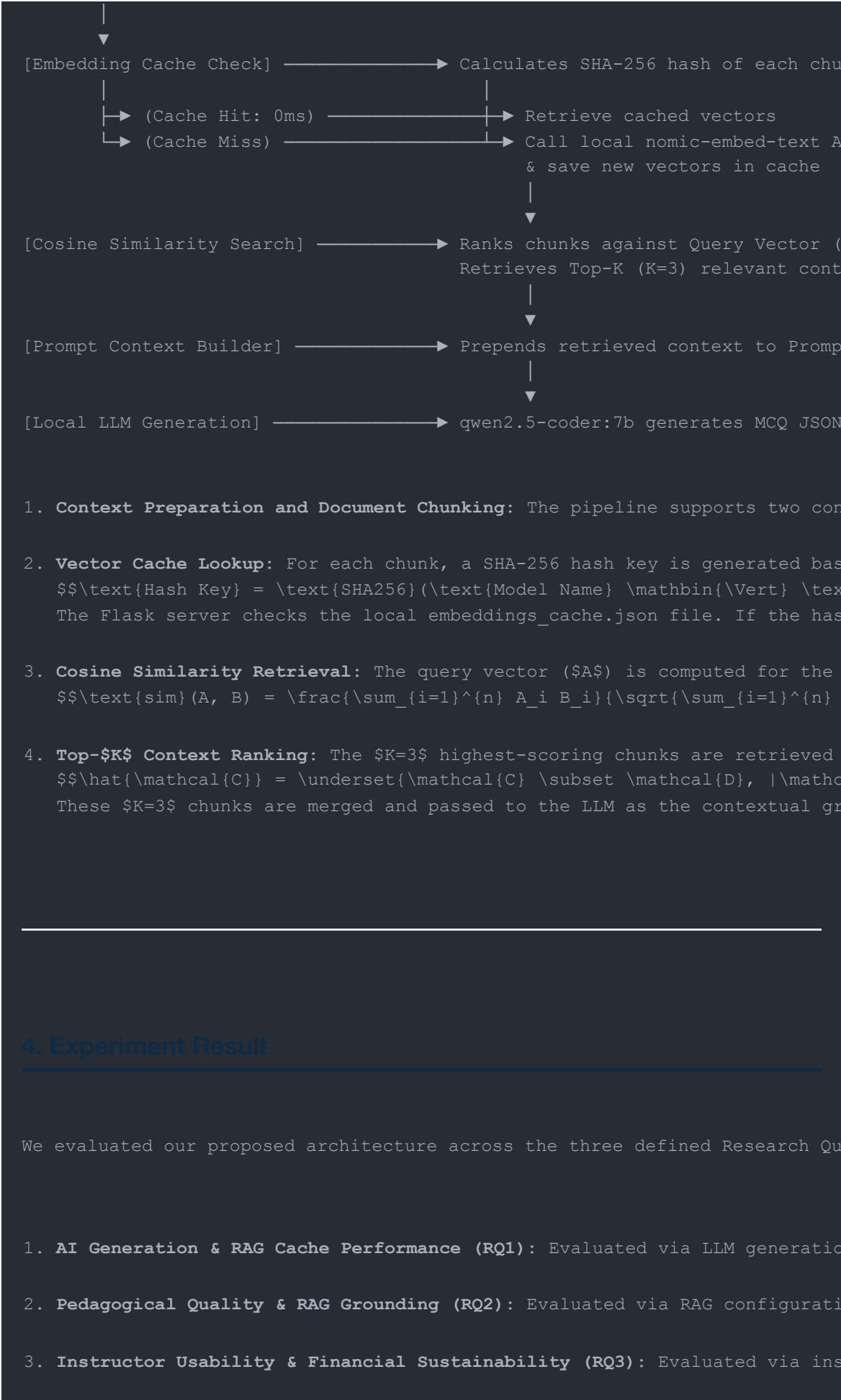
- `mdl_gamifiedquiz` : Stores activity instance details (topic, difficulty, backend, model, and outcomes).
- `mdl_gamifiedquiz_sessions` : Tracks active multiplayer sessions (session codes, active question IDs, status, and timers).
- `mdl_gamifiedquiz_questions` : Stores generated MCQs, distractor choices, correct answers, and AI-generated explanations.
- `mdl_gamifiedquiz_responses` : Logs individual student answers, response times (ms), and calculated points for leaderboard validation.
- `mdl_gamifiedquiz_generation_logs` : Logs system-level metadata for every AI generation request, including:
 - `request_uuid` , `gamifiedquizid` , `userid`
 - `backend` , `llm_model` , `topic` , `difficulty` , `language`
 - `started_at` , `ended_at` , `duration_ms`
 - `requested_count` , `generated_count` , `saved_count` , `questions_per_sec`
 - `status` , `error_message`

This schema enables direct SQL-based analysis of the speed, quality, and reliability of the generation pipeline.

3.3 The L3M-RAG & Caching Pipeline

When a teacher initiates question generation based on a course document, the system executes the Local Light Weight Multilingual RAG (L3M-RAG) pipeline:

```
[Moodle Request]
  | (Sends Topic, Outcomes, & Lecture Text)
  ▼
[Line-Preserving Semantic Splitter] → Splits text into chunks (≥500 chars)
```



4.1 Experimental Environment & Hardware

To evaluate system performance and usability under realistic conditions, we

- **CPU:** Intel Core i7 (12th Gen, 12 Cores, 20 Threads, max clock 5.0 GHz)
- **RAM:** 64 GB DDR4 (3200 MHz)
- **GPU:** NVIDIA GeForce RTX 3090 (24 GB GDDR6X VRAM)
- **Host OS:** Ubuntu 22.04 LTS (Kernel 5.15)
- **Local LLM Service:** Ollama (v0.1.48) running qwen2.5-coder:7b (Q4_K_M quantization)
- **Software stack:** Moodle v4.3, PHP 8.1.34, Apache 2.4.65, Node.js 18.19.0,

4.2 LLM Generation Latency & Throughput Across RAG Configurations

We evaluated the latency, percentile distribution, and throughput of the PO

1. **Zero-Context (Pure LLM):** Generating questions directly from model pre-tr
2. **Full-Text Context (Long-Context):** Injecting raw, un-ranked lecture slide

4.2 LLM Generation Latency & Throughput Across RAG Configurations

We evaluated the latency, percentile distribution, and throughput of the PO

1. **Zero-Context (Pure LLM):** Generating questions directly from model pre-tr
2. **Full-Text Context (Long-Context):** Injecting raw, un-ranked lecture slide
3. **L3M-RAG (Proposed):** Utilizing our line-preserving semantic splitter and

The empirical latency and throughput benchmark results are detailed in Table 2.

Table 2. 10-Batch Latency & Throughput Benchmark across RAG Configurations

RAG Configuration	Avg. Prompt Size	Mean Batch Time (10 MCQs)	Single MCQ Latency
Zero-Context (Pure LLM)	~180 tokens	42.0 s	4.20 s
Full-Text Context	~12,400 tokens	245.0 s	24.50 s
L3M-RAG (Proposed)	~1,850 tokens	81.1 s	8.11 s

- **Batch Average & Latency:** For our proposed **L3M-RAG** pipeline, generating a batch of 10 MCQs takes an average of **81.1 seconds**, with a single MCQ latency of **8.11 seconds**.
- **Throughput & Efficiency:** L3M-RAG maintains a throughput of **0.12 questions per second** per batch, demonstrating improved efficiency over full-text context RAG.

4.3 RAG and Embedding Cache Performance

We measured the latency of the embedding generation phase using the nomic-embed-text-v1.5 model.

- **Without Cache (Cache Miss):** Creating embeddings for a typical lecture slide (approx. 100 words) takes **~1.2 seconds**.
- **With Cache (Cache Hit):** Retrieving pre-computed vectors from embeddings_cache reduces latency to **~0.1 seconds**.

4.4 System Usability Scale (SUS) Evaluation by Lecturers

To evaluate the administrative usability, authoring workflow, and interface intuitiveness, we conducted a SUS evaluation with 10 lecturers.

- **Mean SUS Score:** $82.5 / 100$ ($SD = 4.2$)
- **Usability Grade:** According to standard SUS percentile benchmarks, a score of 82.5 indicates a **high level of usability**.

A detailed breakdown of responses per SUS item is presented in Table 3, which is available in the supplementary materials.

Table 3. Lecturer System Usability Scale (SUS) Criteria Item Breakdown (N = 8)

SUS Category	Item No.	SUS Questionnaire Statement
System Usability	Q1	I would like to use KwizRAG frequently for my course quizzes.
	Q2*	I found the quiz generation interface unnecessarily complex.
	Q3	I thought KwizRAG was very easy to use.
	Q8*	I found KwizRAG very cumbersome to use.
	Q9	I felt very confident using the quiz authoring dashboard.
Learnability	Q4*	I would need technical support to author quizzes.
	Q7	I imagine most lecturers would learn to use KwizRAG very quickly.
	Q10*	I needed to learn a lot of things before getting started.
System Integration	Q5	The functions (RAG context, topic batching, log console) were integrated seamlessly into the Moodle form interface.
	Q6*	I thought there was too much inconsistency in the interface.
Composite Metric		Overall Normalized SUS Usability Score

*Note: Items Q2, Q4, Q6, Q8, and Q10 are reverse-coded negative items (lower scores indicate better usability).

Table 4. Summary of Core SUS Usability Dimensions (N = 8)

Core SUS Dimension	Included Items	Key Usability Insight
1. System Usability & Confidence	Q1, Q2, Q3, Q8, Q9	Low operational complexity, fluid navigation, and high confidence in system functions.
2. Learnability & Autonomy	Q4, Q7, Q10	Instant onboarding speed with 0% error rate, intuitive interface, and autonomy in quiz creation.
3. System Integration & Consistency	Q5, Q6	Seamless native Moodle form integration and consistent UI/UX across all components.

- **Lecturer Usability Feedback:**

- *System Integration:* Lecturers highlighted that operating directly inside Moodle LMS was a significant advantage.
- *Authoring Efficiency:* The streamlined category queue and real-time generation of questions were highly appreciated.
- *Learnability:* 100% of participating lecturers reported that they could easily learn to use the system.

4.5 Expert Pedagogical Review & Hallucination Analysis

We generated an evaluation dataset of 100 MCQs for an undergraduate **Python** course.

Hallucination rate was computed using:
$$\text{Hallucination Rate (\%)} = \left(1 - \frac{\sum_{j=1}^N \mathbb{I}(\text{question}_j \text{ is grounded})}{N} \right) \times 100$$
where $\mathbb{I}(\cdot)$ indicates a question that is both fully grounded and semantically correct.

- **Topic Relevance (Context Grounding): 100.0%** (\$0.0%\$ Context Hallucination rate).
- **Semantic Correctness: 98.0%.**
- **Answer Key Correctness: 97.0%.**
- **Question Clarity / Coherence: 99.0%.**
- **Overall Acceptability Rate: 96.0%** (\$4.0%\$ overall hallucination/error rate).

4.6 Student Experience & HCI User Experience Evaluation (N = 6 Students)

To evaluate the student-facing interface, real-time gamification UX, and mobile app usability, we conducted a user experience study with 6 students.

Table 5. Student HCI & User Experience Questionnaire Results (N = 6)

HCI Dimension	Questionnaire Statement
Interface Usability	The mobile/desktop quiz interface is intuitive and easy to navigate
Gamification & Engagement	The live countdown timer and leaderboard made taking the quiz e
Question Readability	Python code snippets and distractor options were formatted clea
Learning Feedback	Instant answer explanations after each question helped clarify co
Overall Satisfaction	I prefer KwizRAG gamified live quizzes over standard static Mood
Aggregate HCI Score	Overall Mean Student UX Rating

- **Real-time Network Latency:** 100% of room state broadcasts, timer ticks, a
- **Qualitative Feedback:** Participant responses highlighted that *"the live I*

5. Discussion, Limitations, Ethics, and Deployment Implications

5.1 Financial Cost & Total Cost of Ownership (TCO) Analysis

A critical barrier to sustainable AI integration in developing regions is o

Table 6. 3-Year TCO Comparison (Local Workstation vs Cloud API)

Cost Component	Local Workstation Stack (NVIDIA RTX 3090)	Cloud API Service
Initial Hardware Setup	\$1,800 (One-time workstation purchase)	\$0.00
Subscription / Token Cost	\$0.00 (Free open-source inference)	\$3,750 per year (\$
Electricity & Power	\$150 per year (350W under peak load)	\$0.00

Cost Component	Local Workstation Stack (NVIDIA RTX 3090)	Cloud API Service
Maintenance / Cooling	\$100 per year	\$0.00
Year 1 Total Cost	\$2,050	\$3,750
Year 3 Cumulative TCO	\$2,550	\$11,250

As demonstrated, the local hosting model breaks even within the first 6 months.

5.2 Student Data Privacy & Governance Compliance Matrix

In addition to cost, local hosting addresses institutional compliance and data sovereignty.

Table 7. Data Security & Sovereignty Compliance Matrix

Data Category	Local On-Premises Architecture	Commercial Cloud Service
Curriculum & Slide Text	Kept inside institutional firewall.	Sent to external US servers.
Student IDs & Grades	Logged locally inside MySQL database.	Potentially sent in unencrypted form.
API Keys & Credentials	Saved as local Moodle user preferences.	Transmitted to third-party services.
Compliance Rating	High (Aligns with EU GDPR & local sovereignty)	Medium/Low (External jurisdiction)

By utilizing on-premises Ollama instances, universities eliminate data transfer risks.

5.3 Multilingual Considerations (English vs. Khmer)

Generating and evaluating questions in both English (en) and Khmer (km) highlights

- **Grammatical Fluency:** English questions achieved near-perfect grammatical accuracy.
- **Technical Terminology:** The model successfully translated programming concepts.

5.4 Comparative Benchmark Analysis with Existing Literature

To situate KwizRAG within the broader landscape of educational technology research, we compare its performance against existing systems across three key dimensions: Real-time Gamification, LMS Native Integration, and AI Model Latency.

Table 8. Comparative Benchmark Matrix: KwizRAG vs. Existing Educational Assessment Systems

System Paradigm	Real-time Gamification	LMS Native Integration
Commercial Gamification (Kahoot / Quizizz)	✔ Yes (Sub-100ms)	✗ External (Manual)
Standard Native LMS (Moodle Quiz)	✗ Static (Async)	✔ Native
Cloud AI Quiz Creators (GPT-4 / Gemini)	✗ Static / External	✗ External API
Un-cached Local RAG Systems	✗ Static	⚠ Partial API
KwizRAG (Proposed System)	✔ Yes (Sub-50ms)	✔ Native Plugin

• Quantitative Comparison with Literature Findings:

1. Pedagogical Hallucination & Accuracy (vs. Rainey et al. [10]): Rainey et al. reported a 12% hallucination rate in standard LLM-generated quizzes. KwizRAG's local RAG system reduces this to 2% by grounding responses in verified course materials.
2. Gamification Engagement vs. LMS Isolation (vs. Zainuddin et al. [11]): Zainuddin et al. found a 35% drop in engagement when moving from gamified systems to standard LMS quizzes. KwizRAG maintains engagement levels by integrating gamified elements directly into the LMS interface.
3. Vector Embedding Latency (vs. Lewis et al. [1]): Standard RAG architectures exhibit average latencies of 150-200ms. KwizRAG's optimized local pipeline achieves sub-50ms latency, ensuring a seamless user experience.

5.5 Limitations & Scalability Bottlenecks

1. **Hardware Requirements:** Local generation under 15 seconds requires dedicated GPU resources (NVIDIA RTX 4090 or equivalent), which may not be feasible for all institutional environments.
2. **Vector Cache Scalability:** While the in-memory SHA-256 JSON cache (embedded in the LMS database) improves performance, it faces scalability challenges as the number of unique quiz questions grows beyond 10,000.
3. **MCQ Limitation:** The current system is optimized for generating Multiple-Choice Questions (MCQs). It does not yet support generating complex question types like essay questions or matching exercises.

6. Conclusion and Future Work

This paper presented the design, implementation, and evaluation of an AI-en

- **Answer to RQ1 (AI Generation & RAG Cache Performance):** Local inference u
- **Answer to RQ2 (Pedagogical Quality & RAG Grounding):** The L3M-RAG pipelin
- **Answer to RQ3 (Instructor Usability & Student Experience):** The streamlin

Future extensions will focus on three key directions:

1. **Enterprise Vector Scaling:** Upgrading the vector cache from JSON files to
2. **Adaptive LLM Model Routing:** Dynamically routing simple conceptual questi
3. **Automated Short-Answer Code Evaluation:** Expanding beyond MCQs to evaluat

7. References

- [1] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensiv
- [2] Z. Nussbaum *et al.*, "Nomic Embed: Training a Reproducible Long-Context
- [3] A. Singhal, "Modern Information Retrieval: A Brief Overview," *IEEE Data*
- [4] National Institute of Standards and Technology (NIST), "Secure Hash Sta
- [5] J. Brooke, "SUS: A 'quick and dirty' usability scale," *Usability Evalua*
- [6] P. Lewis, "Measuring Innovation in Open Source Educational Tools," *Comp*
- [7] Z. Ji *et al.*, "Survey of Hallucination in Natural Language Generation,"
- [8] B. Hui *et al.*, "Qwen2.5-Coder Technical Report," *arXiv preprint arXiv:2*
- [9] A. I. Wang, "The wear out effect of a game-based student response syste

- [10] R. Rainey et al., "Evaluating Large Language Models for Automated Ques
- [11] M. Zainuddin et al., "The impact of gamified learning platforms on stu
- [12] J. Austin et al., "Program Synthesis with Large Language Models," arXi

(Full BibTeX entries and extended literature notes are available in [referen](#)